

A Visual Odometry Pipeline

Jacqueline Ulken
University of Zurich
jacqueline.ulken@uzh.ch

Gaurav Kapoor
Zurich University of Applied Sciences
kapu@zhaw.ch

Abstract—The goal of this project was to develop a monocular visual odometry (VO) pipeline designed for pose estimation and landmark triangulation. The pipeline was tested on three datasets: the Parking dataset, KITTI 05, and Malaga. The VO pipeline consists of two components: an initialization module that extracts initial landmarks and a continuous operation module that tracks keypoints, estimates the camera pose and regularly triangulates new landmarks.

The pipeline demonstrated reliable performance across all datasets, with visualizations showcasing smooth trajectories and successful landmark triangulation. However, it suffers from significant scale drift in later frames.

I. INTRODUCTION

This project aims to implement a monocular visual odometry pipeline for pose estimation and landmark triangulation in three different datasets:

a) Parking: a monocular video sequence captured in a parking lot environment. It provides simple scenes with minimal dynamic objects which makes it well-suited for initial testing of the pipeline [1].

b) KITTI 05: part of the KITTI odometry benchmark for autonomous driving research, with stereo video sequences recorded from a vehicle. It includes real-world scenarios, such as moving objects and varying lighting conditions [2].

c) Malaga: stereo imagery collected from urban driving environments. It presents complex urban scenes with dynamic elements [3].

The pipeline consists of two components: initialization and continuous operation. The initialization extracts a set of 2D - 3D correspondences to initialize the camera poses and landmarks. During continuous operation, each frame is processed to estimate the current pose and triangulate new landmarks.

This implementation is based on the recommendations from the VO project statement provided by the Robotics and Perception Group at the University of Zurich [1]. Additionally, many of the exercises from the course were used as references to guide the implementation of key components of the pipeline.

II. PIPELINE OVERVIEW

A. Initialization

The pipeline is initialized by manually selecting two frames (the first and another, sufficiently distant second frame) from the start of the dataset. Keypoint correspondences between the frames are established using the Kanade-Lucas-Tomasi (KLT) tracker.

Next, the relative pose between the two frames is estimated using two-view geometry, and a point cloud of 3D landmarks

is triangulated from the corresponding keypoints. RANSAC is employed to filter out outlier keypoints. The resulting inlier keypoints and their associated landmarks are then used in the continuous operation of the pipeline.

B. Continuous operation

Continuous operation begins with the second frame in the dataset and processes each frame sequentially. For each incoming frame, keypoints from the previous frame are tracked using the KLT (Kanade-Lucas-Tomasi) method. This step establishes correspondences that link the current frame to previously triangulated landmarks. Using these correspondences, the current camera pose is estimated by solving the Perspective-n-Point (PnP) problem with RANSAC, filtering outliers. During tracking, some keypoints and corresponding landmarks will get lost. Therefore the pipeline regularly triangulates new landmarks, by keeping a history of candidate keypoints and triangulating them once sufficient parallax is achieved.

III. IMPLEMENTATION DETAILS

This visual odometry pipeline was developed in Python and the complete source code can be accessed on GitHub: VO Pipeline GitHub Repository. Please refer to the README for details on how to run the code to reproduce our results.

For most algorithms, we chose to rely on the established implementations by the OpenCV library [4]. In this document, OpenCV functions are prefixed with `cv2.` and details about them are taken directly from the OpenCV documentation.

A. Keypoints

To select new keypoints, we use OpenCV's `cv2.goodFeaturesToTrack()` function, which determines strong corners in an image using the Shi-Tomasi corner detector. Existing keypoints are tracked in each frame using OpenCV's implementation of the Kanade-Lucas-Tomasi (KLT) tracker `cv2.calcOpticalFlowPyrLK()`. Untracked keypoints and points that lie outside of the image are filtered out.

Each keypoint is associated with a 3D landmark, so tracking the keypoints to the next frame enables matching the new keypoints with the existing landmarks.

B. Pose Estimation

During initialization, the pose of the second reference frame is estimated by first finding the essential matrix with `cv2.findEssentialMat`. This function will automatically apply RANSAC and we only keep the inlier keypoints

going forward. Then, using the essential matrix we estimate the pose with `cv2.recoverPose()`. However, this pose is only used to triangulate the initial landmarks and is then discarded. We initialize the pipeline by setting the pose corresponding to the first frame as the identity matrix.

Once the pipeline has some initial landmarks, they can be used for further pose estimation. During continuous operation, we use the tracked keypoints of the current frame together with the corresponding landmarks in the function `cv2.solvePnP()` for Perspective-n-Point (PnP) pose computation, while again filtering keypoints with RANSAC.

C. Triangulation

Given two sets of matched keypoints and their corresponding poses, we can triangulate new 3D landmarks. For this, we first compute the two projection matrices using the camera intrinsic matrix and the pose. These are then used in the function `cv2.triangulatePoints()` to triangulate the 3D landmarks.

D. Candidate Keypoints

As the camera moves, landmarks will eventually leave the frame. Therefore we must regularly triangulate new landmarks. In each frame, new candidate keypoints are found, again using Shi-Tomasi corner detection as described in subsection III-A. Only sufficiently different points from existing candidate keypoints are kept.

The first time a new candidate keypoint is observed, its coordinates as well as the current pose of the camera are saved. Then, in subsequent frames, the candidate keypoint is tracked while keeping its first observation coordinates and first observation pose.

Whenever the number of landmarks falls below a certain threshold, we check if any of the tracked candidate keypoints and their first observations have gained enough parallax to yield satisfactory triangulations. For this, we simply compute the Euclidean distance between each candidate and its first observation. If this distance is large enough, the two points as well as the first observation pose and the current frame's pose are used to triangulate a new landmark as described in subsection III-C. The pipeline further checks if this new landmark falls behind the camera. If it does, it is discarded.

Finally, any successfully triangulated candidate keypoints together with their associated landmarks are added to the list of current keypoints and landmarks and are removed from the list of candidates.

E. Visualization

To judge the qualitative performance of the pipeline, we created videos for each of the datasets, visualizing the keypoints and trajectory during the continuous operation.

These visualizations were created with the Matplotlib library in Python and saved as videos using OpenCV. To create the visualizations, the visual odometry pipeline was run on a machine equipped with an Intel processor running at 4.2 GHz,

16 GB of RAM, and a multiprocessor configuration supporting multiple threads.

Each visualization includes four plots:

- a picture of the current frame with all keypoints and candidates,
- a plot of the trajectory of the last 20 frames together with the landmarks, shown from a top-down perspective depicting the x and z axis,
- a graph showing the number of average landmarks and candidates (averaged over the last 20 frames),
- and a plot of the global trajectory.

Figure 1 shows an example of such a visualization.

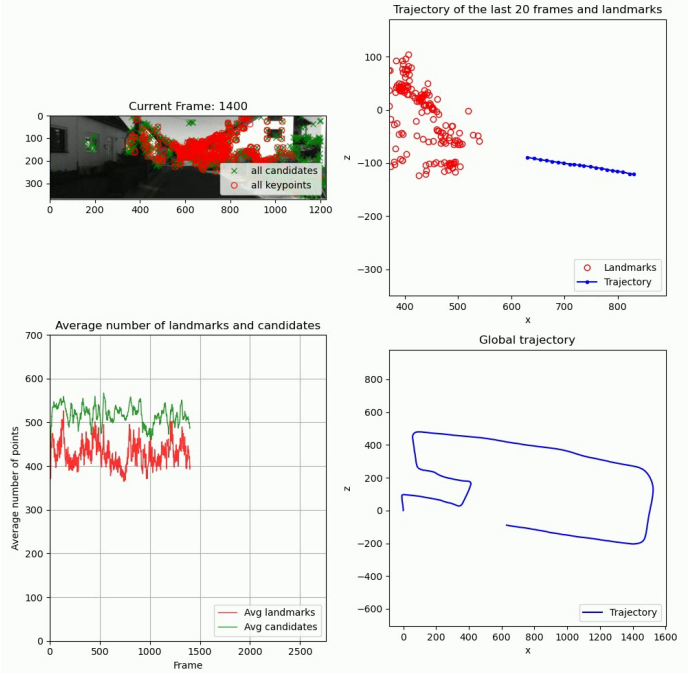


Fig. 1. Visualization of the pipeline performance with the KITTI 05 dataset at frame 1400.

IV. RESULTS AND EVALUATION

For a demonstration of the visual odometry pipeline, refer to the following playlist: VO Pipeline Playlist.

Initially, we followed the project outline's recommendation and, for each candidate keypoint, we calculated the angle between the bearing vectors of the first observation and the current observation, to then decide if the angle was big enough to achieve a high-quality landmark triangulation (angle method). However, we noticed that after a while, the pipeline was losing landmarks more quickly than it was gaining them, which eventually would lead to the failure of the pipeline. We experimented with various parameters and set the threshold angle very small until we finally decided to switch to a more naive validation. Simply checking if the candidate keypoints have moved some minimum distance between frames yielded the best results so far (simple method). This also had the added benefit of being much cheaper to compute. Therefore

we decided to stick with this simple validation in the final version of the pipeline.

The output VO trajectories were evaluated from the popular evo library [5]. This library was used for both plotting and the error estimation between the ground truth and the estimated VO trajectories. The library asks as input the estimated and the reference (ground truth) trajectories and calculates the absolute and the relative pose errors between them. In our case, and as taught in the lectures, we can only track the camera coordinates up to a scale due to the scale ambiguity. So we only know the direction of the translation but not its absolute value.

Here we are therefore considering RPE (relative pose error) for error estimation. The RPE collects all the frame pairs in the two compared trajectories which have a displacement d , in our case 1.0 m. There is also a tolerance given to this distance which in our case is 0.1 m. Then the relative transformation between the estimated and the ground truth trajectory is compared to give an error estimate. We are only considering translation errors as it is a strong enough metric as discussed in [6].

A. Parking Dataset

Here we see in the image frames and also the ground truth data that the displacement of the camera is lateral only, that is, only in the x-axis. So for the error metrics too we are only considering here the displacement in the x-axis. The y- and z-axis are ignored in the RPE computation.

If we consider the angle between the two rays for error computation the error growth is shown in figure 2. On the other hand, if we use the simple distance metric the error is shown in figure 3.

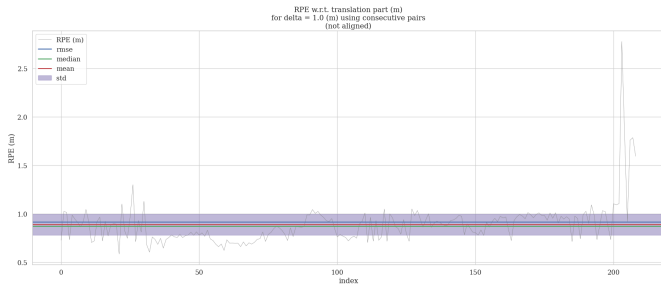


Fig. 2. Growth of the RPEs in the parking dataset for the angle case

In figure 4 we can see that the RPEs of the simple case are lower than that of the angle case. So for relative pose estimation the simple metric is better than the angle metric.

This is also confirmed by the box plot in figure 5. In this we also see some outliers which can be filtered out.

As the trajectory is unidirectional we can simply scale it by the distance ratio between the estimated VO and the ground truth trajectories. Now we can calculate the absolute translational error between the two trajectories. This is performed to implement the global scenario in which we only have a single camera for position estimation. Once the scaling factor

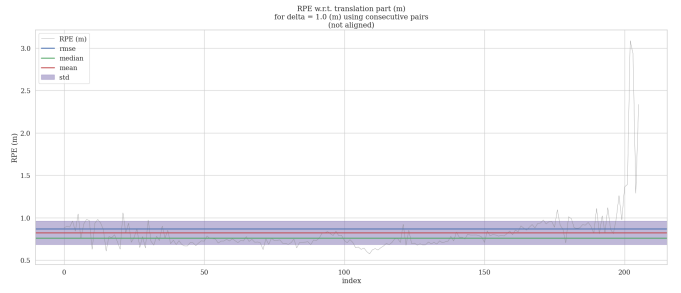


Fig. 3. Growth of the RPEs in the parking dataset for the simple metric

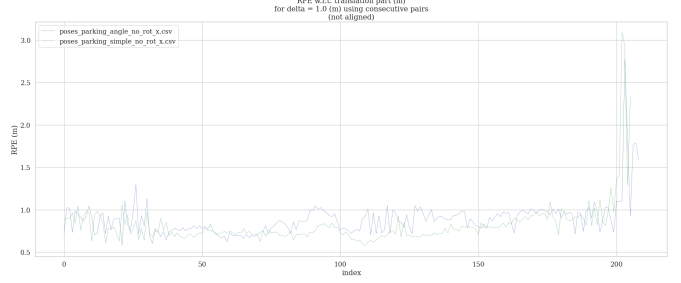


Fig. 4. The growth of the RPEs with time for the parking dataset in the simple (green) vs the angle case (blue)

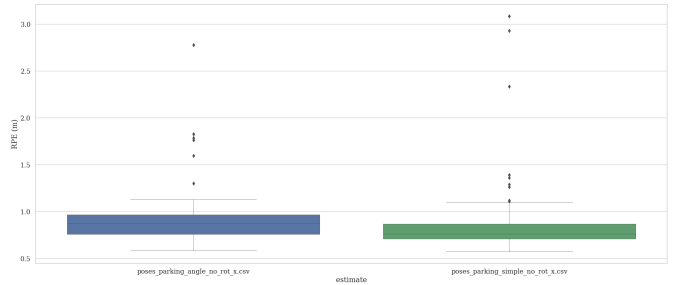


Fig. 5. Box plot comparing the RPEs from the simple (green) and the angle case (blue)

is known, one can estimate the absolute positions of the camera for global localization. After computing this metric we can see in figure 6 that now the angle metric is better than the simple one in case of the APEs.

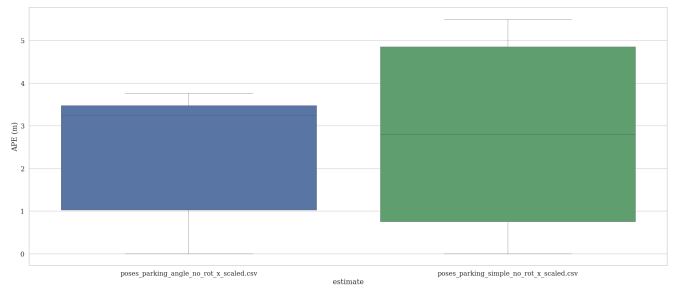


Fig. 6. APEs compared for the scaled trajectories in the simple (green) and the angle case (blue) for the parking dataset

B. KITTI Dataset

The KITTI dataset initially gave us good results but then due to the scale drift the trajectories were showing astronomical numbers. So for evaluating the KITTI dataset trajectory we are only considering the first 200 frames in the trajectory as shown in figure 7. Here we are showing the simple case.

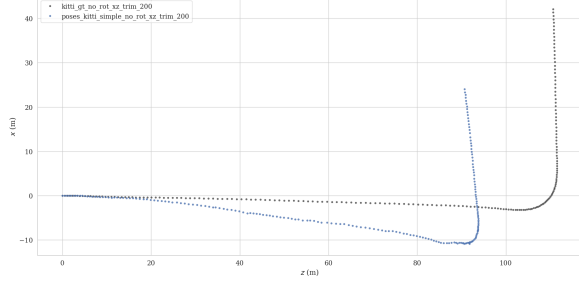


Fig. 7. Trajectory for the KITTI dataset for the first 200 frames in the simple case. Black is the ground truth and the blue is the estimated VO trajectory.

In the RPE in figure 8 we can see the RMSE error of 0.39 m and a mean error of 0.35 m with a standard deviation of 0.17 m, min error of 0.05 m, max errors of 0.85 m which are quite good.

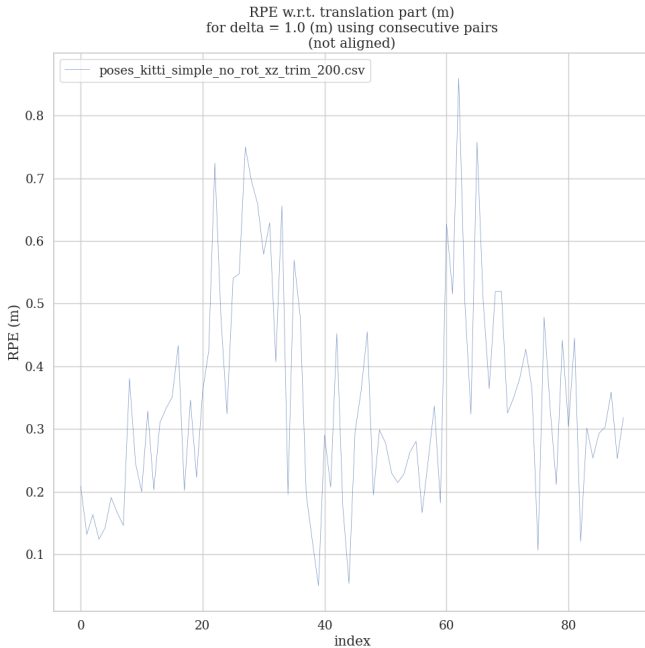


Fig. 8. RPEs growth from the first 200 frames of the KITTI dataset

In the angle case we are not estimating the errors as the trajectory went completely wrong within the first 100 frames only. See figure 9.

C. Malaga Dataset

In the KITTI and the parking dataset the ground truth and our VO, the trajectories were in the camera frame considering positive x-axis to the right and positive z-axis in front of

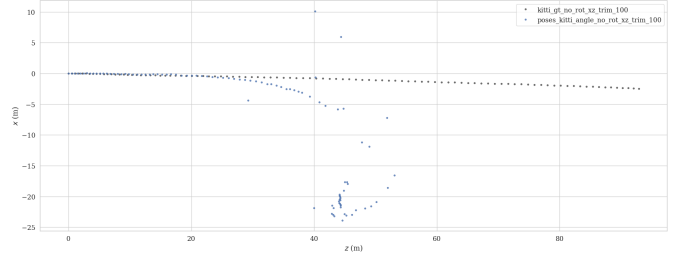


Fig. 9. First 100 frames of the KITTI dataset VO trajectory (blue) compared with the Ground Truth (GT) in the angle case

the camera in a right-hand coordinate system. In the Malaga dataset the ground truth was considered as the local frame trajectories from the GPS coordinates at 1 Hz frequency compared to camera data frequency of 20 Hz. Here the x-axis was considered as the frame in the forward driving direction and the z-axis upwards in the right-hand coordinate system. So we had to transform the trajectory from the xz to the xy plane for plotting and evaluation. The Evo tool again helped in this because of its inbuilt feature of transforming the trajectories based on the provided translation and quaternion rotation.

The plot for the Malaga dataset is shown in figure 10. Here it can be seen that only after a few seconds the trajectory from the VO went completely wrong. Here the trajectory is timestamped as the GPS and the camera data have timestamps.

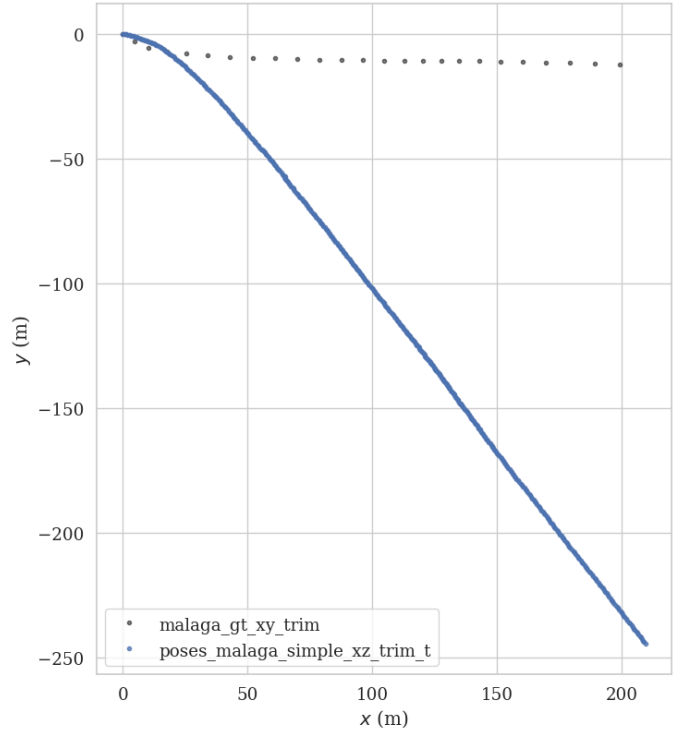


Fig. 10. Malaga dataset trajectory in the simple case

D. Summary of Result

Table I summarizes the error metrics.

TABLE I
RPE METRICS FOR VISUAL ODOMETRY PIPELINE

Metric	Parking [m]	KITTI 05 (first 200 frames) [m]
RMSE	0.87	0.39
Mean	0.82	0.35
Median	0.76	0.32
Std. Dev.	0.27	0.17
Min	0.57	0.05
Max	3.09	0.86

In the error metric table we can see that the RPEs from the KITTI dataset are lower than the Parking dataset. Although the Parking dataset performed much better with the developed VO algorithm. Here one should note that we are only considering the first 200 image frames for the KITTI dataset for the RPE estimation instead of the complete 599 frames in the parking dataset.

V. CONCLUSION

In this project, we successfully developed a VO pipeline capable of keypoint tracking, pose estimation and landmark triangulation. The pipeline achieved locally consistent trajectories across the three datasets, but suffered significant scale drift.

The results from the parking dataset showed that the simple distance-based candidate triangulation method performed better than the angle-based approach in terms of relative pose error. On the KITTI 05 dataset, the pipeline initially produced good results for the first 200 frames, achieving an RMSE of 0.39 m and a mean error of 0.35 m. However, scale drift became a significant issue in longer trajectories.

Future improvements could focus on integrating bundle adjustment and loop closure to mitigate scale drift, as well as extending the pipeline to support additional datasets or real-time applications.

AUTHOR CONTRIBUTIONS

Jacqueline Ulken – Developed the initial working implementation of the visual odometry (VO) pipeline, including the initialization and continuous operation modules. Jacqueline was responsible for writing and integrating the core components of the pipeline, including the main module, the state and the dataset classes.

Gaurav Kapoor – Reviewed the initial implementation, correcting mathematical errors and addressing issues in the code. Gaurav created the visualization module, and conducted the quantitative evaluation of the pipeline using LibreOffice Calc and the Evo tool as mentioned in the report.

Both authors collaborated closely on debugging and testing the final pipeline.

REFERENCES

- [1] Robotics and Perception Group University of Zurich, “Mini-project: A visual odometry pipeline,” 2024. Available at <https://rpg.ifi.uzh.ch/teaching.html>.
- [2] A. Geiger, P. Lenz, and R. Urtasun, “KITTI vision benchmark suite,” 2012. Available at <http://www.cvlibs.net/datasets/kitti/>.
- [3] J.-L. Blanco, J. Moreno, and J. Gonzalez-Jimenez, “The malaga urban dataset,” 2014. Available at <http://www.mrpt.org/MalagaUrbanDataset>.
- [4] G. Bradski, “The OpenCV Library,” *Dr. Dobbs’s Journal of Software Tools*, 2000.
- [5] M. Grupp, “evo: Python package for the evaluation of odometry and slam.” <https://github.com/MichaelGrupp/evo>, 2017.
- [6] M. Grupp, Z. Zhang, and D. Scaramuzza, “Icra17 zhang reproducible workshop.” <https://github.com/MichaelGrupp/evo>, 2017.

VI. TRAJECTORY EVALUATION UPDATE

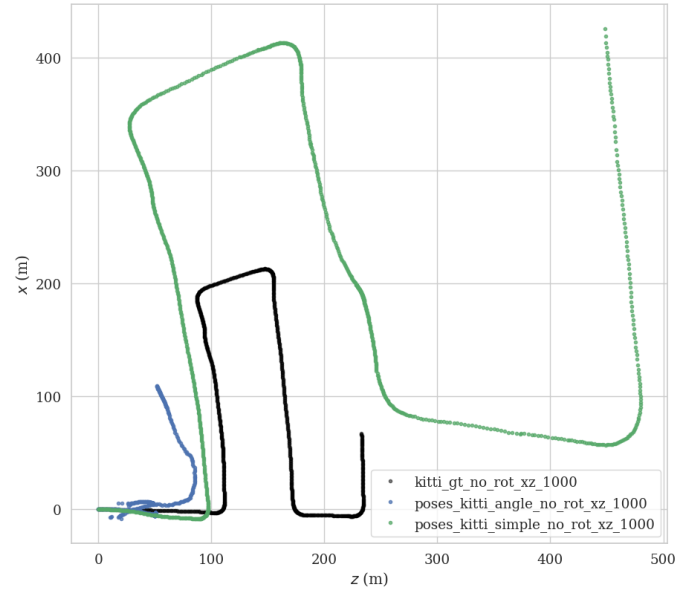


Fig. 11. VO from the first 1000 frames from the KITTI dataset computed on Jacqueline’s PC. Green is the simple metric, blue is the angle one and the black one is the ground truth trajectory

Although the VO algorithm is deterministic for the same parameters, it showed different results on both the authors’ PCs. In Gaurav’s PC the results were drifting apart and going to really huge numbers in the trajectories after few frame computations. And even after running the algorithm again we had same results which was also expected due to the deterministic nature of the algorithm. Jacqueline’s PC had better and more plausible results.

As the task of the quantitative evaluation of the trajectories was assigned to Gaurav, the errors discussed in the results section IV correspond to the one run on Gaurav’s PC. The cause of the error is unknown but could be something to do with different numpy library behavior or the overall Python installation.

Also, this calculation difference with the trajectories on the two PCs was unknown until a random visual check of trajectory files was performed due to suspicion. So, here we are showing you some of the significant improvements made by the trajectories from Jacqueline’s PC for the KITTI dataset.

364	-47.729884	364	284.624196
365	-15.405957	365	285.596982
366	712.061662	366	286.47259
367	698.930209	367	287.47966
368	680.868748	368	288.505572
369	585.130807	369	289.731692
370	534.133614	370	290.758934
371	409.182228	371	291.626477
372	-0.911039	372	292.63184
373	19426.237594	373	293.368015
374	262.86095	374	294.417209
375	10127.076397	375	295.416769
376	9434.277534	376	296.055084
377	8413.485681	377	296.914877
378	8457.538726	378	297.809549
379	7841.539386	379	298.688933
380	7417.115287	380	299.602037

Fig. 12. X-axis trajectory comparison between Gaurav’s PC (left) and Jacqueline’s PC (right) with the simple metric from the KITTI dataset

In Fig. 11 we can see that the trajectory from the simple metric in Green look similar in shape to the ground truth trajectory in black but has a significant scale drift. Also the VO trajectory now looks much more accurate than shown in Fig. 7 and Fig. 9.

The new RPEs from the KITTI dataset are shown in table II. Although the errors seem to be higher now than shown earlier in table I but one should keep in mind that earlier we were only considering the first 200 frames of the KITTI dataset and now we are considering 1000 for the error computation.

TABLE II
UPDATED RPE METRICS FOR VISUAL ODOMETRY PIPELINE

Metric	Parking [m]	KITTI 05 (first 1000 frames) [m]
RMSE	0.91	1.45
Mean	0.89	1.13
Median	0.89	0.84
Std. Dev.	0.17	0.9
Min	0.32	0.05
Max	2.04	6.28

The magnitude of the trajectory difference in the 2 PCs is shown in Figure 12. Here both the trajectories are from the KITTI dataset’s simple metric and show the x-axis of the current camera frame. On the left, we can see that Gaurav’s PC (left) is showing completely different and much larger results than Jacqueline’s (right) for the same camera frame. Based on Jacqueline’s results, this was a strong reason to perform the evaluation again.